

Learning Objectives

This lab lets you practice `for` and `while` loops (including branching in the loops)

Estimated Size

2 program files: `leap_year.py`, `positive_numbers.py`

Setup

In your Python files, you must include author information. On the first line, add a `# Author comment line`, where you mention your full name. Similarly, include `your email and Spire ID` on the `# Email` and `# Spire ID` comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author    : John Snow
# Email     : jsnow@umass.edu
# Spire ID  : 12345678
```

1. Implement `is_leap_year` (5 points)

A leap year is defined as:

- Years divisible by **400** are leap years.
- Years divisible by **100 but not 400** are **not** leap years.
- Years divisible by **4 but not 100** are leap years.
- All other years are not leap years.

Write a function, called `is_leap_year`, inside `leap_year.py`, which should:

- Take a single integer parameter `year`
- Return boolean `True` if the year is a leap year, otherwise return boolean `False`

Important notes:

- The return type of your function must be **boolean**, it may **NOT** be a string.
- You can use if-else statements, but it is also possible to return the result directly using a boolean expression.
- Remember to use the modulo operator `%` to check divisibility.

Below are examples of running a correct implementation of this function:

```
print(is_leap_year(2000))    # True (divisible by 400)
print(is_leap_year(1900))    # False (divisible by 100 but not 400)
```

```
print(is_leap_year(2012))    # True (divisible by 4 but not 100)
print(is_leap_year(2019))    # False (not divisible by 4)
```

2. Implement `list_leap_years` (5 points)

Now that you can check whether a single year is a leap year, let's extend this idea. Write a function, called `list_leap_years`, inside `leap_year.py`, which should:

- Take two integer parameters `a` and `b` (with `a <= b`)
- Use a **while loop** to check every year between `a` and `b` (inclusive)
- Return a list of all years that are leap years in that range

Important notes:

- You must use a **while loop** in your implementation.
- You may call your `is_leap_year` function from the previous problem to check each year.
- Return a **list of integers**, not strings.
- If no leap years are found, return an empty list `[]`.

Below are examples of running a correct implementation of this function:

```
print(list_leap_years(1995, 2005))
# [1996, 2000, 2004]

print(list_leap_years(1896, 1905))
# [1896, 1904]

print(list_leap_years(2019, 2021))
# [2020]

print(list_leap_years(2013, 2015))
# []
```

3. Implement `filter_positive` (5 points)

Write a function, called `filter_positive`, inside `positive_numbers.py`, which should:

- Take a list of integers as a parameter
- Use a **for loop** to check each element in the list
- Collect only the positive numbers (greater than `0`) into a new list

- Return the new list

Important notes:

- You must use a **for loop** to build the list.
- You may not use list comprehensions for this problem.
- The return type of your function must be a **list of integers**.
- If there are no positive numbers, return an empty list `[]`.

Below are examples of running a correct implementation of this function:

```
print(filter_positive([1, -3, 5, 0, -2, 7]))
# [1, 5, 7]

print(filter_positive([-5, -1, -10]))
# []

print(filter_positive([10, 20, -30, 40]))
# [10, 20, 40]
```

4. Submit your code to Gradescope

Log in to [Gradescope](#), select **Lab 05: Loops (Code)**, and submit **all files**. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure, it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions, which will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the [UMass Academic Honesty Policy and Procedures](#) so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is

academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty, you should re-read the course policies. We assume you have read the course policies in detail, and by submitting this project, you have provided your virtual signature in agreement with these policies.